



CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Weeks 1–2 • Day-by-Day Companion

Beginner • Python • PowerShell on Windows • 2 hrs/day • with Anthropic Academy mapping

SCOPE	Days 1–14 of 112 • Foundations phase
FOCUS	Claude API • Prompt engineering • Structured output
ACADEMY	Building with the Claude API • Modules 1 & 2 + selected M3 lessons
SHELL	PowerShell on Windows • paths use backslashes
VERSION	v2 • with Academy mapping per day

DAY-BY-DAY COMPANION • WEEKS 1–2

TOC

14 Days

- 1 **Week 1 • Monday** — Setup & First API Call
- 2 **Week 1 • Tuesday** — Message Structure & Multi-Turn Chat
- 3 **Week 1 • Wednesday** — Content Blocks & Vision
- 4 **Week 1 • Thursday** — Streaming
- 5 **Week 1 • Friday** — Parameters That Matter — temperature, max_tokens, stop_sequences
- 6 **Week 1 • Saturday** — Token Counting & Cost Awareness
- 7 **Week 1 • Sunday** — Consolidation & First Weekly Review
- 8 **Week 2 • Monday** — Prompt Engineering Basics
- 9 **Week 2 • Tuesday** — Few-Shot Prompting & Measurable Accuracy
- 10 **Week 2 • Wednesday** — Structured Output — JSON the Right Way
- 11 **Week 2 • Thursday** — Pydantic — From JSON to Typed Python Objects
- 12 **Week 2 • Friday** — The Validation Retry Loop
- 13 **Week 2 • Saturday** — Prompt Caching
- 14 **Week 2 • Sunday** — Week 2 Review

Each day pairs a hands-on lab with the specific Anthropic Academy lessons that cover the same theory (gold blocks). Both are free; watching is optional but recommended — the videos are the official source the exam draws from. Hyperlinks throughout are functional.

WEEK 01

Monday

Day 1 of 112

Setup & First API Call

INTENT Move from *claude.ai* user to API developer. End the day with a working Python script that talks to Claude.

00-10 Warm-up

Open a notebook (paper or digital). Write three lines:

1. Today's date and "Day 1 of 112".
2. Your exam target date (today + 16 weeks). Pin this somewhere visible.
3. One sentence: *"By Saturday I want a working CLI chatbot that talks to Claude."*

That's your North Star for the week. Close the notebook.

10-60 Setup — infrastructure first, theory later

Day 1 is unusual: instead of the normal 30/70 theory-vs-lab split, you spend most of it on dev environment so the next 111 days have zero friction. Work through these in order.

1. Install Python 3.11+. Verify in PowerShell with `python --version`. If it prints 3.11.x or higher, skip ahead. Otherwise install from python.org/downloads — tick "Add python.exe to PATH" during install.

2. Install `uv` (Anthropic-recommended Python package manager — far faster than pip):

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close PowerShell, reopen it, then verify with `uv --version`.

3. Get an Anthropic API key. Go to console.anthropic.com → sign up → Settings → API Keys → Create Key. Copy it now (you'll never see it again). Settings → Billing → add \$5–10 of credits. That covers all 16 weeks of labs.

4. Create the prep repository. On GitHub, create a new *private* repo called `cca-prep`. Clone it locally:

```
cd C:\Users\<you>\Desktop
mkdir cca-prep
cd cca-prep
```

(Or clone with `gh repo clone <you>/cca-prep` if you have the GitHub CLI.) Then initialize the project:

```
uv init
uv add anthropic python-dotenv
mkdir labs\week-01
mkdir notes
```

5. Store the API key safely. Create `.env` in the repo root:

```
New-Item .env -ItemType File
New-Item .gitignore -ItemType File
```

Open `.env` in any editor and add this single line (paste your real key):

```
ANTHROPIC_API_KEY=sk-ant-...
```

Open `.gitignore` and add:

```
.env
.venv/
__pycache__/*
*.pyc
```

Never commit the key. If you ever paste it into a chat or push it to a public repo by accident, immediately rotate it in the Anthropic console.

+30

► Watch on Anthropic Academy

The official course covering today's material is *Building with the Claude API* on [Anthropic Academy](#) (free, ~8h total, registers with email). You don't need to watch all of it today — just the lessons that match today's lab.

From Module 1 • Getting Started with Claude:

- **Reading** • Getting an API key (~10 min)
- **Reading** • Setup your development environment (~15 min) — Anthropic's preferred setup; compare with what you just did
- **Video** • Overview of Claude models (~4 min)
- **Video** • Making a request (~6 min)

If you prefer notebooks to videos, the same content is in Jupyter form at github.com/anthropics/courses. Read or watch — pick one — and don't double-dip.

60-110 Lab — your first API call

Create `labs\week-01\hello_claude.py` with this content:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "In one sentence, what is the Model Context Protocol?"}
    ],
)

print(response.content[0].text)
```

Run it from the repo root:

```
uv run python labs\week-01\hello_claude.py
```

You should see a single-sentence answer in the terminal. If you hit an error, read the message — most likely it's one of these:

- **401 / authentication_error** — your `.env` isn't being read or the key is malformed. Confirm the file is in the repo root (same folder as `pyproject.toml`) and the line has no quotes around the value.
- **model_not_found** — the model name has changed. Check docs.claude.com → [Models overview](#) for the current daily-driver Sonnet ID and update the script.
- **insufficient credits** — top up in Settings → Billing.

Once it works, modify the script three times and re-run each time to feel the API:

1. Change `model` to `"claude-opus-4-7"` and re-run. Notice the latency difference. Opus is slower and pricier but more capable.
2. Add `system="You answer in exactly five words."` as a parameter to `client.messages.create()`. Re-run. Observe how a system prompt steers behavior.
3. Change `max_tokens` to `50` and ask Claude to "Write a paragraph about the history of Python." Re-run. Watch it get truncated mid-sentence.

110-120 **Capture**

Create `notes\01-api-foundations.md` with 3-5 sentences answering: *what did I just see?* Focus on what surprised you. Then commit:

```
git add .  
git commit -m "day 1: first api call works"  
git push
```

You should end Day 1 with: working dev environment, one committed Python file that calls Claude successfully, a notes file with your thoughts. That's it. **Day done.**

END OF DAY A working `hello_claude.py` is committed and pushed. You can call Claude from your machine.

WEEK 01

Tuesday

Day 2 of 112

Message Structure & Multi-Turn Chat

INTENT Understand the message-list mental model. Build a 3-turn CLI chat loop.

00-10 Warm-up

Re-read your `notes\01-api-foundations.md` from yesterday. In your head, answer: "What is the difference between `system` and a `user` message?" If you're hazy, that's fine — you'll know cold by end of day.

Open the project in your editor:

```
cd C:\Users\<you>\Desktop\cca-prep
code .
```

(Or open it in whatever editor you use. If you're not using VS Code yet, Day 2 is a good day to install it — `winget install Microsoft.VisualStudioCode`.)

10-40 Theory — how Claude sees a conversation

Claude is stateless between API calls. Every `messages.create` request is independent — Claude has no memory of previous requests. To carry on a conversation, *your code* stores the history and re-sends it every turn.

That's why the `messages` parameter is a list, not a string. A 3-turn conversation looks like this:

```
messages = [
  {"role": "user", "content": "What is 2+2?"},
  {"role": "assistant", "content": "4."},
  {"role": "user", "content": "And times 10?"},
]
```

Two roles only: `user` (you, or your app's user) and `assistant` (Claude). They must **alternate** — never two user messages in a row, never two assistant messages in a row. Always end on a user message; Claude's response becomes the next assistant turn that you append.

System prompts are separate. A `system` is not a message in the list — it's its own top-level parameter:

```
client.messages.create(
  model="claude-sonnet-4-6",
  max_tokens=1024,
  system="You are a terse math tutor. Answer in one number, no explanation.",
  messages=[{"role": "user", "content": "What is 2+2?"}],
)
```

System sets persistent behavior. Use it for role, tone, constraints. It applies to every turn, not just the next one.

Read this short section in the docs (10 minutes): [docs.claude.com → Build with Claude → Overview](https://docs.claude.com/build-with-claude/overview). Note any term you don't know — write it on your daily page to come back to.

+30

► Watch on Anthropic Academy**From Module 1 • Getting Started with Claude** ([Skilljar](#)):

- **Video** • Multi-turn conversations (~9 min) — exactly the pattern in today's lab
- **Video** • Build a simple chat bot (~4 min)
- **Video** • System prompts (~6 min)
- **Video** • Exercise on writing a system prompt (~1 min)

~20 minutes total. Watch these *before* the lab if you learn better by seeing an implementation first; watch *after* if you'd rather build then compare.

40-110 **Lab — 3-turn CLI chat loop**

Create `labs\week-01\chat_loop.py`:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

SYSTEM = "You are a friendly Python tutor. Keep answers under 3 sentences."
history = []

print("Chat with Claude. Press Ctrl-C to exit.")

while True:
    try:
        user_input = input("You: ").strip()
        if not user_input:
            continue

        history.append({"role": "user", "content": user_input})

        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            system=SYSTEM,
            messages=history,
        )

        assistant_text = response.content[0].text
        history.append({"role": "assistant", "content": assistant_text})

        print(f"
Claude: {{assistant_text}}
")

    except KeyboardInterrupt:
        print("
Bye.")
        break
```

Run it:

```
uv run python labs\week-01\chat_loop.py
```

Now **actually test the memory**. Have this conversation:

1. You: "My name is Marco."
2. You: "What's 2 + 2?"
3. You: "What did I say my name was?"

Claude should answer "Marco" on turn 3 — because you sent the full history. Then **remove the line** `history.append({"role": "assistant", "content": assistant_text})` and re-run with the same conversation. Claude won't remember your name anymore, because nothing is preserving its replies. *That* is what statelessness means in practice.

Bonus exercise (do if you have time): track and print the cumulative token cost after each turn using `response.usage.input_tokens` and `response.usage.output_tokens`. You'll see input grow on every turn — because you're re-sending the whole history. That cost growth motivates everything you'll learn about context management in Week 9.

110-120 Capture

Append to `notes\01-api-foundations.md` a section titled "Statelessness and the messages list". Three sentences max. Then:

```
git add .
git commit -m "day 2: chat loop with history"
git push
```

END OF DAY `chat_loop.py` runs and Claude remembers your name across turns. You can explain (in your own words) why removing the assistant-append line breaks that.

WEEK 01

Wednesday

Day 3 of 112

Content Blocks & Vision

INTENT Understand that response and request content are *LISTS* of typed blocks, not strings. Send an image to Claude.

00-10 Warm-up

Look back at `hello_claude.py` line 13: `print(response.content[0].text)`. Today you find out why it's `content[0]` and not just `content`.

Pick any photo on your computer — a screenshot, a meme, a vacation snap. Anything. Move a copy into `labs\week-01\` and rename it `test_image.png` (or .jpg).

10-40 Theory — content blocks

The Claude Messages API uses **content blocks** on both sides of the conversation. Every message's `content` field can be either a plain string (shorthand) or a list of typed block objects (full form).

Block types you'll meet by Week 4:

- `text` — plain text. The one you've used so far.
- `image` — a base64-encoded image (or a URL). Sent on user turns.
- `document` — a PDF, sent base64-encoded. Useful in Week 2.
- `tool_use` — Claude's request to call a tool. You'll see these in Week 3.
- `tool_result` — your code's reply with the tool's output. Week 3.
- `thinking` — Claude's extended-thinking blocks on supported models. Later.

A multi-block user message:

```
{
  "role": "user",
  "content": [
    {"type": "image", "source": {"type": "base64", "media_type": "image/png", "data": ""}},
    {"type": "text", "text": "What's in this image?"},
  ],
}
```

And a typical assistant response object:

```
Message(
  content=[
    TextBlock(type="text", text="It looks like a screenshot of a terminal."),
  ],
  ...
)
```

That's why we wrote `response.content[0].text` — index 0 of a list, then read the block's text field. For a text-only response there's exactly one block. As soon as tool use enters the picture, there can be several.

+30

► Watch on Anthropic Academy

The course's vision content lives in Module 3 (mostly tool use, which is Week 3 for you). Today, watch only:

- [Video](#) • Image support (~10 min) — Module 3 • Claude Features and Tool Use
- [Video](#) • PDF support (~1 min) — same module; you'll need this in Week 5 for Project 1

Skip everything else in Module 3 for now. Tool use is the entire focus of Week 3 and the videos will land better after a week of structured-output practice.

40-110 Lab — send Claude an image

Create `labs\week-01\vision.py`:

```
import base64
from pathlib import Path
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# Read the image and base64-encode it
image_path = Path("labs/week-01/test_image.png")
image_data = base64.standard_b64encode(image_path.read_bytes()).decode("utf-8")
media_type = "image/png" # change to image/jpeg if your file is .jpg

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "base64",
                        "media_type": media_type,
                        "data": image_data,
                    },
                },
                {
                    "type": "text",
                    "text": "Describe this image in detail. Then tell me one thing about it I might have missed.",
                },
            ],
        },
    ],
)

print(response.content[0].text)
```

Run:

```
uv run python labs\week-01\vision.py
```

Read the output carefully. Claude is genuinely seeing the image — colors, layout, text in screenshots, even small details. **Try three variations:**

1. Take a screenshot of any code in your editor, run the script with that screenshot. Ask: "Explain what this code does."
2. Photograph a handwritten to-do list (or write one and snap it). Ask: "Transcribe this to-do list as a JSON array of strings."
3. Use a chart or graph image (any from a Google Image search saved locally). Ask: "Summarize the data trends in this chart."

Vision is one of the highest-leverage Claude capabilities — Project 1 in your portfolio will lean on it for invoice extraction.

110-120 **Capture**

Append to your notes: "When does *response.content* have more than one block?" One sentence answer. Commit and push.

END OF DAY Three vision experiments run. You can explain content blocks without looking at notes.

WEEK 01

Thursday

Day 4 of 112

Streaming

INTENT Understand why streaming exists, when to use it, and how to consume a stream in Python.

00-10 Warm-up

Open `claude.ai` in your browser. Send any message and watch the response appear word-by-word. That's streaming. Today you'll do the same thing from Python.

10-40 Theory — why streaming

Two reasons to use streaming:

1. **Perceived latency.** A user staring at a blank screen for 8 seconds feels slow. A user watching text appear after 400ms feels fast — even if the total response time is the same. Every chat UI you'll build later uses streaming for this reason.
2. **HTTP timeouts.** For long responses (large `max_tokens`), the API recommends streaming because a single long-held HTTP connection without any bytes flowing can time out. Streaming keeps the connection alive with server-sent events.

The Python SDK exposes streaming via `client.messages.stream(...)`, which is used as a *context manager* (a `with` block). Inside, you iterate over either `stream.text_stream` (just the text deltas — what you usually want) or the full event stream (when you need block-start, block-stop, tool-use markers, etc.).

Read the official streaming docs (15 minutes): [docs.claude.com → Streaming](https://docs.claude.com/streaming). Focus on the difference between `text_stream` and iterating events.

+30 ▶ Watch on Anthropic Academy

From Module 1 • Getting Started with Claude (Skilljar):

- **Video** • Response streaming (~8 min) — direct match with today's lab

One video. Watch it, compare the instructor's code with yours. The SDK syntax is identical to what you've written.

40-110 Lab — two streaming patterns

Create `labs\week-01\streaming.py`:

```
import time
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

PROMPT = "Write a 200-word vivid description of a thunderstorm at sea."

# --- Pattern 1: non-streaming (baseline) ---
print("=== NON-STREAMING ===")
t0 = time.time()
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": PROMPT}],
)
elapsed = time.time() - t0
print(response.content[0].text)
print(f"
[took {elapsed:.2f}s total, nothing shown until end]
")

# --- Pattern 2: streaming, text only ---
print("=== STREAMING ===")
t0 = time.time()
first_byte_time = None
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": PROMPT}],
) as stream:
    for text in stream.text_stream:
        if first_byte_time is None:
            first_byte_time = time.time() - t0
            print(text, end="", flush=True)
total = time.time() - t0
print(f"

[first byte at {first_byte_time:.2f}s, total {total:.2f}s]")
```

Run it:

```
uv run python labs\week-01\streaming.py
```

You should see two outputs. The first appears all at once after a long wait. The second appears word-by-word almost immediately. The *total* time is similar, but the *perceived* experience is completely different. The `[first byte at]` number is what UX engineers call TTFB — Time To First Byte — and it's the metric that matters for chat UIs.

Bonus: change `max_tokens` to `4096` and re-run. The gap between non-streaming wait time and streaming TTFB will be even more dramatic.

110-120 **Capture**

Note: "*When would I NOT use streaming?*" Answer in one sentence (hint: structured output parsing, background jobs, batch processing). Commit.

END OF DAY You can read a streaming script and explain what's happening byte by byte.

WEEK 01

Friday

Day 5 of 112

Parameters That Matter — temperature, max_tokens, stop_sequences

INTENT *Develop intuition for the knobs you'll turn every day for the next 16 weeks.*

00-10 Warm-up

Make a guess on paper: "At temperature 0, will Claude give the same answer twice to the same prompt?" Write Yes or No. You'll find out today.

10-40 Theory — the three core knobs

temperature (range 0.0 to 1.0, default 1.0). Controls randomness. At 0 the model picks the highest-probability next token every time, making output deterministic-ish (not 100% guaranteed identical, but very close). At 1 it samples more broadly, giving creative or varied output. **Rules of thumb for the exam:**

- Classification, extraction, JSON output, anything requiring a single correct answer → temperature 0.
- Creative writing, brainstorming, marketing copy → temperature 0.7 to 1.0.
- Conversational chat → temperature 0.5 to 0.7 is a fine default.

max_tokens . Hard ceiling on response length. You've already met this. Two production reasons to set it explicitly:

- Cost control — output tokens are 5x more expensive than input tokens. A runaway response is real money.
- Latency control — longer responses take longer; capping **max_tokens** caps the worst case.

It's *required* on every Messages API call. There is no "no limit" option — pick a sensible ceiling.

stop_sequences . A list of strings that, if Claude generates them, immediately stop the response. Used heavily in structured-output extraction: if you wrap JSON in ````json ... ````, you set `stop_sequences=["```"]` so Claude stops the moment it closes the fence, never writing chatty filler after. You'll use this constantly in Week 2.

+30 ► Watch on Anthropic Academy

From Module 1 • Getting Started with Claude ([Skilljar](#)):

- **Reading** • Controlled model output (~10 min)
- **Video** • Temperature (~6 min)
- **Video** • Structured data (~6 min)
- **Video** • Structured data exercise (~5 min)

The Structured data videos are a preview of what you'll deep-dive on Wednesday of Week 2. Watch now — it will pre-load the pattern.

40-110 Lab — temperature A/B and stop_sequences

Create `labs\week-01\temperature_ab.py` :

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

PROMPT = "Write a short opening line for a fantasy novel."

def run(temp: float, n: int = 3):
    print(f"
    --- temperature={temp} ---")
    for i in range(n):
        r = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=100,
            temperature=temp,
            messages=[{"role": "user", "content": PROMPT}],
        )
        print(f"{i+1}: {r.content[0].text}")

run(0.0)
run(1.0)
```

Run:

```
uv run python labs\week-01\temperature_ab.py
```

At `temperature=0` the three opening lines will be nearly identical. At `temperature=1` they'll be wildly different. **This is exam-fundamental.** The choice between temp 0 and temp 1 is the most common parameter question on the test.

Now create `labs\week-01\stop_sequences.py` :

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# Without stop_sequences – Claude tends to add commentary after the JSON.
r1 = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    messages=[{
        "role": "user",
        "content": "Output a JSON array of three random colors as hex codes. Wrap in
    ``json ... ``",
    }],
)
print("--- WITHOUT stop_sequences ---")
print(r1.content[0].text)
```

```
# With stop_sequences=["```] – Claude stops the moment it closes the code fence.
r2 = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    stop_sequences=["```"],
    messages=[{
        "role": "user",
        "content": "Output a JSON array of three random colors as hex codes. Start with
```json",
 }],
)
print("
--- WITH stop_sequences=["```] ---")
print(r2.content[0].text)
print(f"
stop_reason: {r2.stop_reason}")
```

Run it. Notice that in the first version Claude often appends something like "These are vibrant colors that work well together." In the second version the response cuts off cleanly. The `stop_reason` field will be `"stop_sequence"` instead of `"end_turn"` — that's how your code can tell which case happened.

#### 110-120 Capture

Note: write a 5-line table in your notes — Task → Temperature recommendation. Cover: classification, JSON extraction, creative writing, chat, code generation. Commit.

**END OF DAY** You have intuition for temperature and can predict the difference between 0 and 1 responses.

## WEEK 01

## Saturday

Day 6 of 112

## Token Counting &amp; Cost Awareness

**INTENT** *Develop a feel for what API calls actually cost. Build a tool that estimates batch costs before you spend money.*

**00-10 Warm-up**

Open `console.anthropic.com` → Usage . Look at what you've spent so far this week. Probably pennies. By Project 3 (Week 12) you'll be running batch jobs where a single mistake costs \$5. Cost awareness starts now.

**10-40 Theory — tokens, pricing, and caching**

**Tokens, not words.** Claude charges per token. One token is roughly  $\frac{1}{4}$  of an English word — "hello" is 1 token, "antidisestablishmentarianism" is 6 tokens. Numbers, punctuation, and code each tokenize differently. The only reliable count comes from Anthropic's tokenizer, exposed via `client.messages.count_tokens()` .

**Input vs output pricing.** Output tokens cost more — usually 5x more. Current Sonnet 4.6 pricing is approximately \$3 per million input tokens, \$15 per million output tokens. Two implications:

- Long prompts with short answers (classification) are cheap. Short prompts with long answers (writing) are pricier.
- In a chat loop like Tuesday's, *input* grows every turn (you re-send history), which is why long conversations get expensive even when each reply is short.

**Prompt caching.** If you re-send the same long prefix many times (a system prompt, a document, a tool definition list), you can mark it with `cache_control` and Anthropic caches it server-side. Cached input is roughly 10x cheaper. You'll wire this up Saturday of Week 2 — today, just know it exists.

**+30 ► Watch on Anthropic Academy**

The course has no dedicated lesson on cost estimation or token-counting — today's lab is "extra." Use the saved 30 minutes to read one of these instead:

- [Current pricing page on docs.claude.com](#) — confirm the prices in your `cost_estimator.py` are current
- [Token counting docs](#) — explains the `count_tokens` endpoint you just used

## 40-110 Lab — build a batch cost estimator

Create `labs\week-01\cost_estimator.py`:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

Pricing as of May 2026 – verify at https://docs.claude.com/en/about-claude/pricing
PRICE_PER_MTOK = {
 "claude-sonnet-4-6": {"input": 3.0, "output": 15.0},
 "claude-opus-4-7": {"input": 5.0, "output": 25.0},
 "claude-haiku-4-5": {"input": 1.0, "output": 5.0},
}

def estimate_batch(prompts: list[str], model: str, expected_output_tokens: int = 200):
 rates = PRICE_PER_MTOK[model]
 total_input = 0
 for p in prompts:
 # count_tokens gives you the exact token count Anthropic will bill for
 result = client.messages.count_tokens(
 model=model,
 messages=[{"role": "user", "content": p}],
)
 total_input += result.input_tokens
 total_output_est = expected_output_tokens * len(prompts)

 input_cost = total_input / 1_000_000 * rates["input"]
 output_cost = total_output_est / 1_000_000 * rates["output"]
 total = input_cost + output_cost

 print(f"Model: {model}")
 print(f" Requests: {len(prompts)}")
 print(f" Input tokens: {total_input:,}")
 print(f" Est. output toks: {total_output_est:,}")
 print(f" Estimated cost: ${total:.4f} (input ${input_cost:.4f} + output $
{output_cost:.4f})")

Try it
sample_prompts = [
 "Summarize the French Revolution in one paragraph.",
 "Explain why the sky is blue in terms a child would understand.",
 "Write a haiku about debugging.",
] * 10 # pretend you have 30 documents to process

estimate_batch(sample_prompts, "claude-sonnet-4-6", expected_output_tokens=150)
estimate_batch(sample_prompts, "claude-opus-4-7", expected_output_tokens=150)
estimate_batch(sample_prompts, "claude-haiku-4-5", expected_output_tokens=150)
```

Run:

```
uv run python labs\week-01\cost_estimator.py
```

---

You'll see the 5x output-cost ratio in action and the Opus-vs-Haiku tradeoff. Save this script — you'll use the pattern in every project. **Habit:** before any batch over ~10 requests, run an estimate. Surprises cost real money.

**110-120**     **Capture**

Note: "*Why do output tokens cost 5x input tokens?*" Take a guess (hint: it's about generation compute, not storage). Commit. Push.

**END OF DAY**   You can estimate any batch job's cost before running it.



## WEEK 01

## Sunday

Day 7 of 112

## Consolidation &amp; First Weekly Review

**INTENT** Lock in Week 1. Write notes that future-you will actually re-read.

**00-10 Warm-up**

Today is consolidation, not new material. Resist the urge to "get ahead." The point of the weekly review is to convert this week's labs into durable knowledge — without it, you'll have to re-learn this stuff in Week 8.

**10-60 Review — re-read & re-run**

Open `notes\01-api-foundations.md`. Read what you wrote on Days 1-6. Ask yourself: *"Could a beginner read this and understand what I learned?"* If no, rewrite. The notes are for future-you under exam pressure — they must be clear, not impressive.

Re-run each lab from the week, in order. Don't just open the file — actually run it:

```
uv run python labs\week-01\hello_claude.py
uv run python labs\week-01\chat_loop.py
uv run python labs\week-01\vision.py
uv run python labs\week-01\streaming.py
uv run python labs\week-01\temperature_ab.py
uv run python labs\week-01\stop_sequences.py
uv run python labs\week-01\cost_estimator.py
```

If any fails — fix it. If any takes more than 10 seconds to recall what it does — re-read your own notes.

**+30 ► Watch on Anthropic Academy**

**Formal Module 1 self-test** ([Skilljar](#) • Module 1 quizzes):

- **Quiz** • Getting started with the Claude API (~15 min)
- **Quiz** • Optimizing model output (~10 min)

Take both today as your Sunday review's official self-test. If you score below 80% on either, re-watch the videos for the questions you missed before starting Week 2.

---

**60-110 Self-check & Week 2 prep**

---

**Week 1 self-check question** (from the master plan): "What goes in *messages* vs *system* vs *tools*?"

Open `notes\weekly-reviews.md` (new file) and write a one-paragraph answer in your own words. No looking at the labs. If you can't answer cleanly, re-read Tuesday's theory section now and try again.

Then write 3 more sentences:

1. The most surprising thing I learned this week was...
2. The concept I'm least sure about is...
3. One question I'd ask if I could...

Item 2 is your Week 2 priority. Item 3 — bring it to me when we check in mid-Week-2.

**Prep for Week 2:** create the folder.

```
mkdir labs\week-02
```

Week 2 is **prompt engineering and structured output**. You'll learn the patterns Anthropic actually tests on — JSON forcing, retry loops, Pydantic validation. Don't read ahead. Just rest.

---

**110-120 Capture**

---

Final commit of the week:

```
git add .
git commit -m "week 1 complete: API foundations"
git push
```

Then close the laptop. Take the rest of Sunday off. Burnout is the #1 risk on a 16-week plan; rest days are non-negotiable.

**END OF DAY** Week 1 complete. Seven days of consecutive commits in your `cca-prep` repo. You can answer the self-check question without notes.

---

## WEEK 02

## Monday

Day 8 of 112

## Prompt Engineering Basics

**INTENT** Learn the principles Anthropic actually uses internally. Apply them to a measurable task.

## 00-10 Warm-up

Week 2's theme: **get reliable, parseable output from Claude**. By Sunday you'll have a Pydantic-validated classifier with a retry loop — the most exam-relevant pattern in the entire syllabus.

Today: prompt engineering principles, applied immediately.

## 10-40 Theory — the five high-leverage techniques

Read this section: [docs.claude.com](https://docs.claude.com) → [Prompt engineering](#). Internalize these five techniques. Each one shows up in scenario questions.

1. **Be specific.** "Summarize this" is vague. "Summarize this in three bullet points, each under 15 words, focused on financial implications" is specific. Specificity reduces variance.
2. **Give examples (few-shot).** Showing 2-3 worked examples of input→output is almost always more effective than describing the format in prose. Comes up tomorrow.
3. **Use XML tags.** Wrap distinct sections in tags like `<document>...</document>` and `<instructions>...</instructions>`. Claude was trained to attend to XML structure; this measurably improves performance on complex prompts.
4. **Prefill the assistant turn.** You can include an empty (or partial) `assistant` message at the end of `messages`; Claude continues from where you left off. Starting with `{` forces JSON. Starting with `<analysis>` forces structured analysis. This is the most exam-favorite technique on the test.
5. **Chain of thought.** Ask Claude to think before answering — "think step by step in `<thinking>` tags, then give your final answer in `<answer>` tags." For hard reasoning tasks this dramatically improves correctness, at the cost of more output tokens.

## +30 ► Watch on Anthropic Academy

From Module 2 • Prompt Engineering & Evaluation ([Skilljar](#)):

- [Video](#) • Prompt engineering (~11 min)
- [Video](#) • Being clear and direct (~2 min)
- [Video](#) • Being specific (~5 min)
- [Video](#) • Structure with XML tags (~4 min)
- [Video](#) • Providing examples (~7 min)

~29 minutes. The XML-tags video directly justifies the `<email>...</email>` wrapping pattern in today's lab — watch it.

## 40-110 Lab — applying the techniques to one task

Today's task: **classify a customer support email by urgency**. Create `labs\week-02\classifier_v0.py`:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

EMAIL = """Hi,
Our entire payment system has been down for the last 3 hours. Customers are calling. We are
bleeding revenue. Please help ASAP. We pay $5000/month for this service.

– Jamie, COO, Acme Corp"""

v0: naive prompt
def classify_naive(email: str) -> str:
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=100,
 messages=[{"role": "user", "content": f"What is the urgency of this email? Email:
{email}"},
]
)
 return r.content[0].text

v1: specific instructions + constrained vocabulary
def classify_specific(email: str) -> str:
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=20,
 temperature=0,
 system=("You classify support emails by urgency. "
 "Reply with exactly one word from this set: LOW, MEDIUM, HIGH, CRITICAL. "
 "No explanation, no punctuation, no extra text."),
 messages=[{"role": "user", "content": email}],
)
 return r.content[0].text.strip()

v2: XML tags + prefill
def classify_xml_prefill(email: str) -> str:
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=20,
 temperature=0,
 system=("You classify support emails by urgency into one of LOW, MEDIUM, HIGH,
CRITICAL. "
 "Output ONLY the label between tags."),
 messages=[
 {"role": "user", "content": f"{email}"},
 {"role": "assistant", "content": ""},
],
 stop_sequences=[""],
)
 return r.content[0].text.strip()
```

```
print("v0 naive: ", classify_naive(EMAIL))
print("v1 specific: ", classify_specific(EMAIL))
print("v2 xml+prefill: ", classify_xml_prefill(EMAIL))
```

Run it. Watch the three outputs evolve from chatty paragraph → single word → guaranteed-parseable label. v2 is the production-grade pattern: you can do `label = classify_xml_prefill(email)` and trust that `label` is one of four exact strings without any post-processing.

**Bonus:** swap the email for a low-urgency one ("Hi, when you get a chance — could you update my billing address?") and confirm the labels shift accordingly.

### 110-120 Capture

Note the five techniques in your own words. Commit as `day 8: prompt engineering basics`.

**END OF DAY** You've seen the three classifier versions and can articulate why v2 is production-grade.

## WEEK 02

## Tuesday

Day 9 of 112

## Few-Shot Prompting &amp; Measurable Accuracy

**INTENT** Stop guessing whether your prompt is good. Measure it.

## 00-10 Warm-up

Yesterday's classifier *looked* good on one example. Today you measure how good it actually is on 50.

## 10-40 Theory — few-shot and evaluation

**Few-shot prompting** = include worked examples directly in the prompt. The pattern:

```
Example 1:
Input: "Our payment system is down, losing $$$"
Output: CRITICAL

Example 2:
Input: "Can you update my billing address?"
Output: LOW

Now classify:
Input: "{actual_email}"
Output:
```

Few-shot beats zero-shot on most classification, extraction, and formatting tasks — often by 10-30 percentage points of accuracy. The labels in your examples teach Claude both the format AND the threshold (what counts as CRITICAL vs HIGH).

**Why measure?** Prompt engineering without a test set is vibes. You change a word, the output "feels" better, you ship. Two weeks later you discover a regression. The fix: a small hand-labeled **gold set** and an **eval script** that scores any version of your prompt against it. This pattern shows up in every project you'll build, and it's the single most important habit of the next 16 weeks.

## +30 ► Watch on Anthropic Academy

From Module 2 • Prompt Engineering & Evaluation ([Skilljar](#)):

- Video • Prompt evaluation (~2 min)
- Video • A typical eval workflow (~5 min)
- Video • Generating test datasets (~5 min)
- Video • Running the eval (~7 min)
- Video • Code based grading (~7 min)
- Video • Try out a prompt eval (~5 min)

~31 minutes. *Code based grading* describes exactly what your `eval.py` is doing — exact string match against gold labels. *Model based grading* (also in the module, optional today) is the more advanced "LLM as judge" pattern you'll see again in Project 1.

**40-110 Lab — build a 50-example gold set + eval harness**

This lab takes longer than the others — that's expected. Building the gold set IS the lab.

**Step 1.** Create `labs\week-02\gold.jsonl` with 50 lines. Each line is a JSON object: `{"email": "...", "label": "LOW|MEDIUM|HIGH|CRITICAL"}`. Make them up — variety matters more than realism. About 12-13 of each label. Aim for some edge cases: polite-but-urgent ("just wanted to flag that production has been down for 4 hours"), urgent-sounding-but-trivial ("URGENT: can I get a sticker?"), and so on.

(If 50 feels like a lot — it is. Do 30 today and 20 tomorrow as the warm-up. Edge cases are exactly where prompt engineering pays off.)

**Step 2.** Create `labs\week-02\classify.py` with two versions of the classifier — zero-shot and few-shot:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

LABELS = ["LOW", "MEDIUM", "HIGH", "CRITICAL"]

def zero_shot(email: str) -> str:
 r = client.messages.create(
 model="claude-sonnet-4-6", max_tokens=20, temperature=0,
 system=("Classify the urgency. Reply with exactly one of: " + " ".join(LABELS)),
 messages=[
 {"role": "user", "content": f"{email}"},
 {"role": "assistant", "content": ""},
],
 stop_sequences=[""],
)
 return r.content[0].text.strip()

FEW_SHOT_EXAMPLES = """
URGENT: production payments are completely down, losing revenue every minute.
CRITICAL

Hey, when you have a moment, could you update my billing address?
LOW

One of our team members can't log in. Other users are fine. Please look when you can.
MEDIUM

Our admin dashboard has been unreachable for 30 minutes. Three of our staff are blocked.
HIGH
"""

def few_shot(email: str) -> str:
 r = client.messages.create(
 model="claude-sonnet-4-6", max_tokens=20, temperature=0,
 system=(f"Classify the urgency. Reply with exactly one of: {' '.join(LABELS)}").
```

```

"
 f"Examples:
{FEW_SHOT_EXAMPLES}"),
 messages=[
 {"role": "user", "content": f"{email}"},
 {"role": "assistant", "content": ""},
],
 stop_sequences=[""],
)
return r.content[0].text.strip()

```

**Step 3.** Create `labs\week-02\eval.py` :

```

import json
from pathlib import Path
from classify import zero_shot, few_shot

def run_eval(name: str, fn):
 gold = [json.loads(line) for line in Path("labs/week-02/
gold.jsonl").read_text().splitlines() if line.strip()]
 correct = 0
 misses = []
 for item in gold:
 pred = fn(item["email"])
 if pred == item["label"]:
 correct += 1
 else:
 misses.append((item["email"][:60] + "...", item["label"], pred))
 acc = correct / len(gold) * 100
 print(f"
{name}: {correct}/{len(gold)} = {acc:.1f}%")
 print("Misclassifications:")
 for email, gold_label, pred in misses[:10]:
 print(f" [{gold_label} -> {pred}] {email}")

run_eval("zero_shot", zero_shot)
run_eval("few_shot", few_shot)

```

Run:

```
uv run python labs\week-02\eval.py
```

Few-shot should beat zero-shot, usually by 5-20 points. Look at the misclassifications carefully — they're telling you what your examples are missing. Add a covering example, re-run, watch accuracy climb. **That feedback loop is the entire job of a prompt engineer.**

## 110-120 Capture

Note both accuracy numbers in `notes\02-prompts.md` with the date. You'll re-run this script in Week 15 and want to compare. Commit.

**END OF DAY** Two accuracy numbers logged. You've seen few-shot beat zero-shot with your own data.



## WEEK 02

## Wednesday

Day 10 of 112

## Structured Output — JSON the Right Way

**INTENT** Learn the canonical Anthropic pattern for forcing valid JSON output. This is the single most exam-tested technique in the entire syllabus.

## 00-10 Warm-up

If anyone tells you there's a `response_format: json` parameter on the Anthropic API — they're wrong (that's OpenAI). Today you learn the right way.

## 10-40 Theory — the JSON pattern

The Anthropic-canonical pattern for structured JSON output has four parts:

1. **System prompt declares the schema.** Either inline as JSON Schema, or descriptively with field names and types.
2. **System prompt says "respond with valid JSON only, no prose."**
3. **Prefill the assistant turn with {** — Claude continues from that brace, guaranteed to start with valid JSON.
4. **Parse and validate.** Run `json.loads` on the response. If it fails, you retry (Friday's lesson).

You'll layer on Pydantic validation tomorrow. Today: get the prefill working.

## +30 ► Watch on Anthropic Academy

The Skilljar course teaches JSON via Module 3's tool-use mechanism, which is canonically correct but is a Week 3 topic for you. Today, watch this single lesson and skip the rest:

- **Video** • Tools for structured data (~8 min) — Module 3 • Claude Features and Tool Use

You'll see two valid approaches: the prefill pattern (what you built today, in-prompt) and the tool-use pattern (via `tools=[...]`, structurally enforced). Anthropic supports both. Prefill is simpler and exam-cited; tool-use schemas are more production-robust. Week 3 you'll build the tool-use version.

## 40-110 Lab — structured extraction with prefill

Today's task: extract structured info from product reviews. Create `labs\week-02\extract.py`:

```
import json
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

REVIEW = """Bought these for my morning runs. The cushioning is excellent and they're very
lightweight. However the laces came untied constantly and the toe box runs narrow – had to
size up. Overall happy, would recommend to other neutral-foot runners. Paid $130 on sale."""

SCHEMA = """{
 "product_category": string,
 "sentiment": "positive" | "negative" | "mixed",
 "positives": [string],
 "negatives": [string],
 "price_mentioned": number | null,
 "would_recommend": boolean
}"""

def extract(review: str) -> dict:
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=500,
 temperature=0,
 system=(f"Extract structured information from the review. "
 f"Output a single valid JSON object matching this schema:
{SCHEMA}
"
 f"Output JSON only. No markdown, no preamble."),
 messages=[
 {"role": "user", "content": f"{review}"},
 {"role": "assistant", "content": "{}"},
],
)
 # Claude continues from "{", so we must re-add it
 raw = "{" + r.content[0].text
 return json.loads(raw)

result = extract(REVIEW)
print(json.dumps(result, indent=2))
```

Run it. You should see clean JSON. The key trick is the `{"role": "assistant", "content": "{}"}` line — that prefill is what forces Claude to start with valid JSON. Without it, Claude often opens with "Here is the extracted information:" or wraps in markdown fences.

**Now break it on purpose.** Remove the prefill line (the assistant turn with `"{"`) and re-run. You'll often get an error from `json.loads` because Claude added prose. **That** is why we prefill — and why we need a retry loop (Friday).

**Bonus:** try a few more reviews of different products. Note any field Claude struggles with — usually price extraction is tricky when there are multiple numbers in the text.

---

**110-120**   **Capture**

---

Note the four-part JSON pattern in your own words. This is on the exam. Commit.

**END OF DAY** You can extract validated JSON from unstructured text using the prefill pattern.

---

## WEEK 02

## Thursday

Day 11 of 112

## Pydantic — From JSON to Typed Python Objects

**INTENT** Stop trusting raw dicts. Use Pydantic to validate every response against a typed schema.

## 00-10 Warm-up

Yesterday's extractor returned a `dict`. What if Claude hallucinates a field name? What if it returns 'positive' as a list of strings instead of a single value? Pydantic catches these.

## 10-40 Theory — Pydantic v2 in 30 minutes

Pydantic is the de-facto Python library for typed, validated data models. You define a class with type-annotated fields; Pydantic enforces the types at runtime and gives you a useful error if anything's off.

Skim Models and Fields. Don't try to memorize — just get familiar with the shape:

```
from pydantic import BaseModel, Field
from typing import Literal

class Review(BaseModel):
 product_category: str
 sentiment: Literal["positive", "negative", "mixed"]
 positives: list[str]
 negatives: list[str]
 price_mentioned: float | None = None
 would_recommend: bool

Validate a dict against the schema:
data = {"product_category": "shoes", "sentiment": "mixed", ...}
review = Review.model_validate(data) # raises ValidationError if invalid
```

Two things Pydantic gives you that raw `json.loads` doesn't:

- **Type enforcement.** If `sentiment` comes back as "mostly positive" (not in the Literal set), Pydantic raises immediately — you don't have to write the check yourself.
- **Schema export.** `Review.model_json_schema()` gives you a JSON Schema dict you can drop straight into Claude's system prompt. Single source of truth: your Python class.

## +30 ► Watch on Anthropic Academy

The Skilljar course doesn't teach Pydantic directly. Today, read instead:

- Pydantic v2 · Models (~15 min)
- Pydantic v2 · Fields (~15 min)

Don't try to memorize. Get familiar with `BaseModel`, `Field`, type annotations, and `ValidationError`. Everything else you can look up when needed.

## 40-110 Lab — typed extraction

Add Pydantic to the project:

```
uv add pydantic
```

Create `labs\week-02\extract_typed.py`:

```
import json
from typing import Literal
from pydantic import BaseModel, ValidationError
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

class Review(BaseModel):
 product_category: str
 sentiment: Literal["positive", "negative", "mixed"]
 positives: list[str]
 negatives: list[str]
 price_mentioned: float | None = None
 would_recommend: bool

REVIEW = """Bought these for my morning runs. The cushioning is excellent and they're very
lightweight. However the laces came untied constantly and the toe box runs narrow – had to
size up. Overall happy, would recommend to other neutral-foot runners. Paid $130 on sale."""

def extract(review: str) -> Review:
 schema = json.dumps(Review.model_json_schema(), indent=2)
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=500,
 temperature=0,
 system=(f"Extract structured information from the review. "
 f"Output a single JSON object conforming to this JSON Schema:
{schema}
"
 f"Output JSON only."),
 messages=[
 {"role": "user", "content": f"{review}"},
 {"role": "assistant", "content": "{}"},
],
)
 raw = "{" + r.content[0].text
 data = json.loads(raw)
 return Review.model_validate(data) # raises ValidationError if shape wrong

result = extract(REVIEW)
print(result.model_dump_json(indent=2))
```

```
print(f"
Type is: {type(result).__name__}")
print(f"Would recommend: {result.would_recommend} (bool, not string)")
```

Run it. Now you have a typed `Review` object — your IDE autocompletes `result.sentiment`, your type checker catches mistakes, and any deviation from the schema raises immediately rather than failing silently three steps later in your pipeline.

**Test the validation.** Temporarily change the system prompt to ask for `sentiment` to be a list. Re-run. You'll see a clean `ValidationError` from Pydantic explaining exactly what went wrong. *That error message is what you'll feed to the retry loop tomorrow.*

## 110-120 Capture

Note: 'Why does using `Pydantic.model_json_schema()` in the prompt eliminate a whole class of bugs?' Commit.

**END OF DAY** Typed `Review` objects coming back from Claude. `ValidationError` fires cleanly when the schema is violated.

## WEEK 02

## Friday

Day 12 of 112

## The Validation Retry Loop

**INTENT** Build the most exam-tested pattern in the entire CCA-F syllabus. Memorize it cold.

## 00-10 Warm-up

Today's pattern is named in plain English on the exam. If a scenario says "JSON extraction sometimes fails — how should the system recover?" the right answer always involves: **catch `ValidationError`, re-prompt with the error message, cap at N retries**. Build it once today, recognize it forever.

## 10-40 Theory — what makes a good retry loop

Three properties of a production-grade retry loop:

1. **It re-prompts with the actual error.** Don't just retry the same prompt — append the failure to the conversation so Claude can see what went wrong and fix it. This is the difference between "try again and hope" and "try again with corrective feedback."
2. **It has a hard cap (typically 3 attempts).** Without a cap, a malformed prompt can loop forever and drain your credits. With a cap, you fail loudly and route the input to human review.
3. **It distinguishes retryable errors from final failures.** JSON parse errors and validation errors → retry. API errors (5xx, rate limits) → exponential backoff (different pattern, you'll see in Week 9). Auth errors → don't retry, fix the config.

That's the canonical Anthropic pattern. Build it once. Reuse forever.

## +30 ► Watch on Anthropic Academy

No dedicated lesson — the retry-loop pattern is canonical exam material that lives in practice, not in a video. Use the saved 30 minutes to:

- Re-read the [Building Effective Agents](#) essay, specifically the section on evaluator-optimizer loops — your retry loop is a minimal case of that pattern
- Or skim Module 2's optional video *Model based grading* (~10 min) if you skipped it Tuesday

## 40-110 Lab — the retry loop

Create `labs\week-02\extract_with_retry.py`:

```
import json
from typing import Literal
from pydantic import BaseModel, ValidationError
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

MAX_RETRIES = 3

class Review(BaseModel):
 product_category: str
 sentiment: Literal["positive", "negative", "mixed"]
 positives: list[str]
 negatives: list[str]
 price_mentioned: float | None = None
 would_recommend: bool

class ExtractionError(Exception):
 """Raised when extraction fails after all retries."""

def extract(review: str) -> Review:
 schema = json.dumps(Review.model_json_schema(), indent=2)
 system = (f"Extract structured information from the review. "
 f"Output a single JSON object conforming to this JSON Schema:
{schema}
"
 f"Output JSON only, no markdown, no preamble.")

 messages: list[dict] = [
 {"role": "user", "content": f"{review}"},
 {"role": "assistant", "content": "{}"},
]

 last_error = None
 for attempt in range(1, MAX_RETRIES + 1):
 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=500,
 temperature=0,
 system=system,
 messages=messages,
)
 raw = "{" + r.content[0].text

 try:
 data = json.loads(raw)
```



```

 return Review.model_validate(data)
 except (json.JSONDecodeError, ValidationError) as e:
 last_error = e
 print(f" [attempt {attempt}] failed: {type(e).__name__}: {str(e)[:120]}")

 if attempt == MAX_RETRIES:
 break

 # Re-prompt: append Claude's bad output + a corrective user message,
 # then a fresh prefill.
 messages = [
 {"role": "user", "content": f"{review}"},
 {"role": "assistant", "content": raw},
 {"role": "user", "content": (
 f"That output failed validation with: {type(e).__name__}: {str(e)[:200]}."
 "Please re-output a corrected JSON object that matches the schema. "
 "Output JSON only."
)},
 {"role": "assistant", "content": "{}"},
]

 raise ExtractionError(f"Failed after {MAX_RETRIES} attempts. Last error: {last_error}")

REVIEW = """Bought these for my morning runs. Cushioning is excellent. Laces came untied
constantly. Sized up because toe box runs narrow. Would recommend. Paid $130."""

result = extract(REVIEW)
print(result.model_dump_json(indent=2))

```

Run it. On a normal review, you'll get the result on attempt 1 with no retries — good.

**Now force failures.** Temporarily change the system prompt to: *"Output a JSON object with a sentiment field that is a list of strings"* — i.e. ask Claude to produce shape-violating output. Re-run. You'll see attempt 1 fail, the corrective message go out, and (often) attempt 2 succeed. That recovery is the entire pattern.

**Force terminal failure.** Make the system prompt impossible: *"Output an XML document, never JSON"*. Re-run. All 3 attempts fail, the `ExtractionError` fires. In production, that's your signal to route to a human-review queue.

## 110-120 Capture

In `notes\02-prompts.md`, write the retry loop pattern as pseudocode in your own words. No looking at the file. This pattern is on the exam — you want to be able to draw it on a whiteboard. Commit with message `day 12: retry loop locked in`.

**END OF DAY** A retry loop that demonstrably recovers from injected schema violations. Pattern memorized.

## WEEK 02

## Saturday

Day 13 of 112

## Prompt Caching

**INTENT** *Cut your prompt costs by 90% when you have repeated context.*

**00-10 Warm-up**

Imagine you're running yesterday's extractor over 10,000 product reviews. The system prompt (with the JSON schema, the instructions, eventually a few-shot examples) is ~2,000 tokens. It's IDENTICAL across all 10,000 calls. Why pay to send it 10,000 times? You don't have to.

**10-40 Theory — how prompt caching works**

Prompt caching lets you mark a chunk of input — typically the system prompt and any static context — as "cacheable." On the first call Anthropic stores those tokens server-side. Subsequent calls within ~5 minutes (or longer, with extended TTL) hit the cache: those tokens are **roughly 10x cheaper**.

**Three rules of thumb for the exam:**

- **Cache the static prefix.** System prompt, JSON schema, few-shot examples, tool definitions. Anything that doesn't change between calls.
- **Don't cache the variable suffix.** User-specific input goes after the cache breakpoint and is uncached.
- **Caching has a minimum.** The prefix must be at least 1024 tokens (Sonnet) or it won't actually cache. For tiny prompts, caching doesn't help.

You declare a cache point by adding `cache_control={"type": "ephemeral"}` to the last block you want cached. Everything before it (and that block itself) gets cached together.

**+30 ► Watch on Anthropic Academy****From Module 3 • Claude Features and Tool Use ([Skilljar](#)):**

- [Video](#) • Prompt caching (~4 min)
- [Video](#) • Rules of prompt caching (~7 min)
- [Video](#) • Prompt caching in action (~7 min)

~18 minutes. *Rules of prompt caching* covers the 1024-token minimum, the cache scope, and TTL — exam-relevant. Watch all three before your lab if possible.

## 40-110 Lab — measure the savings

Create `labs\week-02\caching.py`:

```
import json
import time
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

A big static system prompt – schema + many examples. Pad to >1024 tokens
so caching actually engages.
LONG_SYSTEM = """You extract structured information from product reviews into JSON.

Output JSON only, matching this schema:
{
 "product_category": string,
 "sentiment": "positive" | "negative" | "mixed",
 "positives": [string],
 "negatives": [string],
 "price_mentioned": number | null,
 "would_recommend": boolean
}

Examples:

These headphones sound amazing but the case fell apart after 2 weeks. $200.
{"product_category": "headphones", "sentiment": "mixed", "positives": ["amazing sound"],
"negatives": ["case fell apart after 2 weeks"], "price_mentioned": 200, "would_recommend":
false}

Perfect coffee maker, brews fast and easy to clean.
{"product_category": "coffee maker", "sentiment": "positive", "positives": ["brews fast",
"easy to clean"], "negatives": [], "price_mentioned": null, "would_recommend": true}

""" + ("More guidance: be thorough. Capture exact phrases for positives and negatives. " * 30)

REVIEWS = [
 "Loved the running shoes. Light, cushioned. Paid $130.",
 "Coffee grinder works but is way too loud. $80.",
 "Watch band broke after a month. Returning. Was $50.",
 "Tent kept me dry through a storm – fantastic.",
 "Backpack zipper failed on day 2. $120 down the drain.",
]

def extract(review: str, use_cache: bool):
 system_blocks = [
 {
 "type": "text",
 "text": LONG_SYSTEM,
 **({"cache_control": {"type": "ephemeral"}} if use_cache else {}),
 }
]
```

```

 r = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=400,
 temperature=0,
 system=system_blocks,
 messages=[
 {"role": "user", "content": f"{review}"},
 {"role": "assistant", "content": "{}"},
],
)
 return r.usage

def run_batch(use_cache: bool):
 label = "WITH CACHE" if use_cache else "WITHOUT CACHE"
 print(f"
=== {label} ===")
 total_input = 0
 total_cache_read = 0
 total_cache_create = 0
 t0 = time.time()
 for rev in REVIEWS:
 u = extract(rev, use_cache)
 total_input += u.input_tokens
 total_cache_read += getattr(u, "cache_read_input_tokens", 0) or 0
 total_cache_create += getattr(u, "cache_creation_input_tokens", 0) or 0
 elapsed = time.time() - t0
 print(f" Total uncached input: {total_input:,}")
 print(f" Total cache reads: {total_cache_read:,}")
 print(f" Total cache creates: {total_cache_create:,}")
 print(f" Elapsed: {elapsed:.2f}s")

run_batch(use_cache=False)
run_batch(use_cache=True)

```

Run it:

```
uv run python labs\week-02\caching.py
```

What to look for: in the cached run, the first request shows `cache_creation_input_tokens > 0` (you're paying a small premium to write the cache). Requests 2-5 show `cache_read_input_tokens > 0` and tiny `input_tokens` — cache hits at 10x cheaper. Total input cost drops dramatically. **Latency also drops** because Anthropic doesn't re-process the cached tokens.

Production rule: any time you're calling the API more than once with the same long prefix, cache the prefix. This is exam-cited and one of the highest-leverage cost optimizations.

## 110-120 Capture

Note your input-token reduction (e.g. '8500 → 1200 tokens, 7x reduction'). You'll cite this number in your Project 1 README. Commit.

**END OF DAY** Measured cache savings on a real batch. You can articulate when to cache and when not to.

## WEEK 02

## Sunday

Day 14 of 112

## Week 2 Review

**INTENT** Lock in. Prep for Week 3 (tool use).**00-10 Warm-up**

You're 1/8 of the way through. Take stock — what stuck, what didn't.

**10-60 Review — re-run & re-test**

Re-run every Week 2 lab:

```
uv run python labs\week-02\classifier_v0.py
uv run python labs\week-02\eval.py
uv run python labs\week-02\extract.py
uv run python labs\week-02\extract_typed.py
uv run python labs\week-02\extract_with_retry.py
uv run python labs\week-02\caching.py
```

For each, ask yourself: "Could I rewrite this from scratch tomorrow without notes?" If no on any, re-read your notes and redo the lab.

**+30 ► Watch on Anthropic Academy**

**Formal Module 2 self-test** ([Skilljar](#) • Module 2 quizzes):

■ **Quiz** • Quiz on prompt evaluation (~15 min)

■ **Quiz** • Quiz on prompt engineering techniques (~15 min)

Take both as your Week 2 official self-test. Aim for 80%+ on both. The Coursera version of this course also has Module 3 quizzes covering tool use and caching — save those for next Sunday after Week 3.

---

**60-110 Self-check & Week 3 prep**

---

**Week 2 self-check question** (from the master plan):

"Why is 'use `response_format: json`' wrong on the Messages API, and what is the correct pattern?"

Write a one-paragraph answer in `notes\weekly-reviews.md`. Bullet the four-part pattern (system prompt + schema + prefill + validation+retry). Don't peek at the labs first — if you can't answer it, you haven't earned Week 2 yet. Re-read Wednesday and Friday and retry.

Then add:

1. Hardest moment this week was...
2. The pattern I want to internalize before Week 9 is...
3. One question I'd ask if I could...

**Prep for Week 3:**

```
mkdir labs\week-03
```

Week 3 is **tool use** — letting Claude take actions in the world (call a weather API, query a database, send an email). You'll build a multi-tool agent by Sunday. The two-week run on prompts + structured output gives you everything you need to design good tools next week.

When you finish Day 14 (today), send me a message: "*Week 2 done.*" I'll write Weeks 3-4 then.

---

**110-120 Capture**

---

```
git add .
git commit -m "week 2 complete: prompts + structured output"
git push
```

Close the laptop. Rest the rest of Sunday.

**END OF DAY** Week 2 complete. 14 days of consecutive commits. You can draw the JSON-extraction pattern from memory.

---